

## RESEARCH ARTICLE

# Advanced GPU Techniques for Dynamic Remeshing and Self-Collision Handling in Real-Time Cloth Tearing

JONG-HYUN KIM<sup>1</sup> AND JUNG LEE<sup>2</sup><sup>1</sup>College of Software and Convergence, Department of Design Technology, Inha University, Michuhol-gu, Incheon 22212, South Korea<sup>2</sup>Department of Computer Engineering, Hanbat National University, Yuseong-gu, Daejeon 34158, South Korea

Corresponding author: Jung Lee (airjung@hanbat.ac.kr)

This work was supported in part by the Hanbat National University in 2023 (Contribution Rate: 20%); in part by the National Research Foundation of Korea (NRF) through the Korean Government (MSIT) (Contribution Rate: 20%) under Grant RS-2023-00254695; in part by the Inha University Research Grant (Contribution Rate: 20%); in part by the Basic Science Research Program through NRF, Ministry of Education (Contribution Rate: 40%) under Grant 2022R1F1A1063180; and in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) through the Korean Government (MSIT), Artificial Intelligence Convergence Innovation Human Resources Development, Inha University (Contribution Rate: 20%) under Grant RS-2022-00155915.

**ABSTRACT** In this paper, we propose a GPU-based framework to accelerate dynamic remeshing and self-collision handling required for tearing thin-shell materials such as cloth: 1) Dynamic remeshing involves updating the shape and connectivity information at the element/node level, making parallel computing challenging and computationally costly. In this paper, we propose a new data structure that enables GPU-friendly calculations. Additionally, 2) to efficiently handle the computationally intensive self-collision handling in tearing cloth, we propose a GPU-based BVH (Bounding Volume Hierarchy) tree construction, update, and traversal method, along with an optimized R-Triangle (Representative-Triangle) technique. We also present a method to optimize kernels based on a complete binary tree in arbitrary triangular meshes, improving performance. Thin-shell materials are utilized in various fields including simulation/animation based on physics, games, surgical simulation, and virtual reality. Tearing cloth requires dynamically updating its geometry and connectivity, which is complex and computationally intensive. This process needs to be performed quickly, especially when dealing with interactive content. Most existing approaches perform remeshing through low-resolution simulations to maintain real-time performance or use pre-divided patterns, which cannot be considered dynamic remeshing and result in low-quality torn surface patterns. In this study, we propose a GPU-optimized dynamic remeshing algorithm that enables real-time processing of high-resolution cloth tearing. Additionally, we introduce a method to accelerate the computationally intensive self-collision handling in tearing cloth simulations using GPUs. To achieve this, we propose a GPU-based complete binary tree method to efficiently construct, update, and traverse a BVH (Bounding Volume Hierarchy) tree and optimize the R-Triangle (Representative-Triangle) technique on triangular meshes using GPUs to minimize primitive collision checks. The proposed method performs dynamic remeshing rather than using predefined split mesh structures, making it suitable for interactive physics-based modeling in applications such as virtual reality surgery simulation, real-time gaming, and virtual environments.

**INDEX TERMS** Thin-shell, cloth simulation, tearing simulation, dynamic remeshing, physically-based simulation, self-collision handling.

The associate editor coordinating the review of this manuscript and approving it for publication was Andrea Bottino<sup>1</sup>.

## I. INTRODUCTION

Attempts are being made to realistically depict the movements of characters and agents to enhance user immersion in virtual environments, utilizing physics-based simulation



**FIGURE 1.** A real photography of torn cloth.

techniques [1]. Among these, cloth simulation influences various industries such as virtual fashion studios, fabric production, animation, and visual effects. However, due to the high computational load and the susceptibility of the shape of the cloth to environmental changes, expressing the effects of tearing the cloth in high-resolution simulations is challenging [2], [3]. In the field of physics-based simulation, research on cloth primarily focuses on the movement of cloth represented based on Continuum Mechanics. Typical models represent cloth as an elastic thin-shell, capturing only its macroscopic geometric features [4]. This model can represent compression, stretching, and bending, allowing various materials to be simulated as cloth. Cloth tearing is represented by the split of the mesh elements due to increasing deformation or direct interaction with external forces such as cutting (see Figure 1). This phenomenon affects not only the remeshing of cloth, but also the simulation of cloth dynamics.

The simulation of cutting deformable bodies has been a continuous area of research in the field of physics-based simulation. It is a core technology in fields such as virtual surgery and is also frequently used in computer animation and games. Generally, virtual cutting in cloth simulation proceeds through the following three processes: 1) the incorporation of cuts into the computational model of the cloth, that is, updating the geometric and topological representation of the simulation domain and numerically discretizing it through governing equations, 2) simulating the cloth surfaces based on this computational model, and 3) finally, performing collision detection and handling.

Most existing approaches are based on the volumetric mesh representation of objects, and when representing thin-shell materials like those used in this study, a triangular mesh structure is used. Early research in this field also utilized tetrahedral meshes, which allows flexible implementation of remeshing when considering cut modeling by splitting elements or snapping element vertices to the cutting plane [5], [6], [7], [8], [9]. However, these methods can produce numerically incorrect-shaped elements, negatively impacting simulation stability. To mitigate this issue, studies have been conducted using regular or semi-regular meshes [10], [11], [12], as well as research using polyhedral meshes [13] and the mesh-free approach based on particles [14], [15], [29].

To accurately represent phenomena such as tearing or fracturing due to external forces, many studies numerically discretize the governing equations using mesh-based approaches such as the Finite Element Method (FEM) [16]. However, as the number of elements in the mesh increases, the computational load becomes very high, making it inefficient. For interactive applications, performance is often enhanced by discretizing space using structures like octrees to efficiently handle geometrical and topological changes caused by cuts, but errors can occur between the discretized spatial meshes, reducing accuracy. Additionally, using heterogeneous structures, which are different simulation domains, is not algorithmically optimal and is not GPU-friendly, making optimization difficult.

In this study, we propose a GPU-based framework for fast and efficient remeshing and self-collision handling in cloth or thin-shell simulations. When the cloth tears, the intersected parts must be remeshed to conform to the cutting shape rather than to the geometric structure of the cloth. The main contributions of this research are the development of a GPU-friendly algorithm to enable real-time high-resolution cloth tearing simulation, and the optimization of BVH tree construction, updating (refitting) and traversal using a complete binary tree on the GPU.

#### Technical contributions

- 1) We propose a framework for implementing tearing cloth with GPU-friendly dynamic remeshing. To minimize the splitting process of elements and nodes, we improve efficiency by passing CutPoint and CutEdge as CutInfo units and in the form of Prefix Sum to the kernel. Each node data is composed of 4 bits, and FaceInfo consists of a total of 12 bits of data. Additionally, the framework allows for the rapid updating of shape and connectivity information in the kernel due to remeshing.
- 2) We propose a GPU framework to accelerate self-collision handling in the tearing cloth process. As the number of triangles can change with each frame during tearing, the computational load for BVH tree construction, updating, and traversal increases. In this paper, we optimize the BVH tree in the form of a complete binary tree to enable GPU-friendly calculations of construction, update, and traversal on triangular meshes. Additionally, we simplify the R-Triangle for GPU calculation, improving the performance of self-collision handling. By efficiently reducing the R-Triangle, we minimize GPU computation time and memory usage, while ensuring stable handling of self-collisions without any missed areas.

## II. RELATED WORK

### A. CLOTH TEARING SIMULATION

In this section, we review representative approaches to expressing dynamic remeshing, focusing on mesh-based or mesh-free cut modeling techniques.

Generally, the virtual cutting of deformable bodies is implemented by handling the geometric and topological representation of the simulation domain. Typically, mesh-based remeshing cuts on the basis of tetrahedral, hexahedral, and polyhedral meshes. In the field of physics-based simulation, triangular meshes are widely used for thin-shells, and tetrahedral meshes for volumetric meshes. In this section, we review studies closely related to these approaches.

There are six main approaches to representing cuts in tetrahedral meshes: element deletion [7], splitting along existing element faces [8], [17], element duplication [18], [19], snapping of vertices [20], element refinement [6], [9], [21], [22], [23], and combined snapping of vertices and element refinement [24].

One of the problems that arises when performing remeshing on deformable bodies is numerical instability in deformation calculations due to the creation of ill-shaped elements. To mitigate this issue, arbitrary-shaped cut surfaces must be accurately represented [25], and the necessary technique for this process is dynamic remeshing. Methods such as element deletion and splitting along existing element faces result in jagged surfaces at the cut. By means of snapping vertices, element refinement, or a combination of both, cuts can be accurately represented. However, since elements are modified in this method, additional procedures are required to prevent the creation of ill-shaped elements. The element duplication method allows the original shape to be maintained on the cutting surface by inserting the accurate surface into the duplicated element. However, this method can create ill-shaped elements, necessitating additional handling techniques.

The easiest way to integrate cuts into a deformable body is to remove the elements that intersect the cutting plane, thereby separating the material. This method is widely used in real-time simulations due to its low computational load [7], but results in low accuracy and visual quality due to jagged surfaces. A solution to this problem is to split the object along the faces of the existing elements [8]. This method works well when predefined cut surfaces are considered through tetrahedralization, but arbitrary direction cuts still produce jagged surfaces. To create new surfaces that match the cutting plane, a method involves snapping vertices onto the cutting surface before splitting the object along the surface of the elements [20].

Therefore, to better represent cutting surfaces, it is necessary to refine the tetrahedra locally. Bielser et al. proposed a 1:17 subdivision method for tetrahedral decomposition by creating vertices on each edge and triangle face [6]. This method has the disadvantage of significantly increasing the number of elements. To minimize this problem, Bielser and Gross subdivided only the edges and faces that are part of the cutting surface [21]. Mor and Kanade proposed a progressive cutting method to minimize the number of newly generated elements [9], and Ganovelli et al. suggested a multi-resolution approach to reduce the number of elements [23]. Bielser et al. also proposed a state machine technique that can track

the topological configuration of each tetrahedron during progressive cutting [22].

Molino et al. [18] proposed the virtual node algorithm to avoid numerical problems caused by ill-shaped elements. The basic idea is to create one or more duplicates of the cut element and associate material properties with the duplicates. Wang et al. [26] improved the virtual node algorithm. This method allows cuts that pass through mesh vertices or are placed on mesh edges and faces (without the need for ambiguous perturbation of the cutting surfaces as in the original version), enables multiple cuts per tetrahedron face, and demonstrates lower algorithmic complexity compared to the method by Sifakis et al. [19]. Additionally, it reduces floating-point rounding errors that occurred in previous techniques, thereby improving the robustness of the algorithm.

In the field of Computer Graphics, Desbrun and Gascuel [27] used a mesh-free method with a particle system to simulate soft material that can split and merge by combining inter-particle forces. Müller et al. [28] proposed a technique for animating volumetric objects based on points. Mesh-free methods have also been applied to offline fracturing [29] and interactive cutting [15]. In addition to deformable bodies, the Moving Least Squares (MLS) method is widely used for the surface reconstruction of point clouds [30], [31].

## B. COLLISION HANDLING

Spatial hashing is a widely used algorithm for collision detection between objects and can be easily parallelized on the GPU [44]. This algorithm has a constant time complexity to query all nearby triangles. Pabst et al. [45] used a regular grid to apply broad phase culling on the GPU for triangle meshes. To handle the non-uniform distribution of geometric primitives in the scene, Faure et al. [46] extended the regular grid to a two-layer grid. Hierarchical grids have been used to further improve the efficiency of spatial partitioning-based collision detection algorithms [46]. While hierarchical grids can address the non-uniform distribution of scene primitives, they do not provide good culling performance for relatively flat materials like cloth models.

Selle et al. [47] proposed a parallel cloth simulation algorithm for multi-core platforms. This method can take up to 30 minutes per frame for a cloth mesh with 500,000 triangles on a 16-core workstation. Tang et al. [48] proposed a streaming algorithm for high-resolution cloth simulation with regular shapes on the GPU. This algorithm was extended for cloth simulation with arbitrary topological structures [49] and can take up to 35 seconds per frame for cloth with 1 million triangles on a typical GPU. These parallel algorithms use BVTT (Bounding Volume Traverse Tree) for parallel collision detection but can result in large memory overhead.

## C. PROBLEM STATEMENT

Cloth simulation has consistently been a topic of interest in the fields of physics-based simulation and animation. Terzopoulos and Fleischer [34] proposed a method to

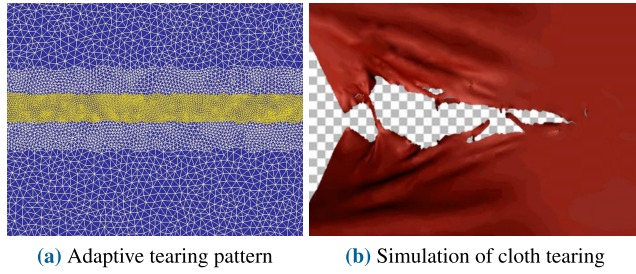


FIGURE 2. Simulation of cloth tearing along pre-defined patterns.

TABLE 1. List of symbols used in this paper.

| Symbol               | Description                                                                                           |
|----------------------|-------------------------------------------------------------------------------------------------------|
| $C_l$ (CutEdge)      | Edges created by truncated borders.                                                                   |
| $C_p$ (CutPoint)     | Point where the CutEdges intersect.                                                                   |
| $C_{info}$ (CutInfo) | Shape information that is cropped from faces and edges. Compute on each Surface and Edge in parallel. |

simulate deformable surfaces such as cloth using the Finite Difference Method (FDM). Since then, many researchers have addressed various aspects of cloth simulation, such as computational efficiency [38], numerical stability [32], [33], collision handling [39], cloth bending [40], and inextensible cloth [41]. Although continuum-based approaches are efficient in representing most of the characteristics of the cloth, they generally include dynamic remeshing processes to simulate tearing of the cloth, which is computationally intensive. For real-time applications, it is challenging to achieve real-time processing without using pre-divided meshes. These approaches, although computationally less demanding because of the reliance solely on mesh resolution for tear patterns, result in low-quality tearing meshes.

In the industry, to avoid this issue, cloth surfaces are often refined with multi-resolution to represent tearing areas in high quality (see Figure 2). However, this approach requires prior knowledge of the tearing regions and is primarily used in static scenes, such as in games, rather than in scenes where tearing occurs in arbitrary directions.

There are many techniques that address the dynamics of cloth simulation, but not many focus on the tearing process. Baraff and Witkin [32] proposed a numerical method to stably represent the motion of cloth simulation. They introduced an implicit integration method to mitigate numerical instability that occurs when using large time-steps. Cloth is represented by a triangular mesh, and the internal force is calculated based on the strain of each triangle. Choi and Ko [33] proposed a stable and responsive semi-implicit cloth simulation technique. The stability provided by this technique allows for the use of large fixed time-steps when simulating all types of fabrics and character motions. Although there are numerous studies on cloth dynamics, this study focuses on techniques for cloth tearing and does not cover dynamics for internal forces.

Terzopoulos and Fleischer [34] introduced methods to tear materials such as paper and cloth. O'Brien and Hodgins

simulated the fracture of brittle objects based on FEM and extended it to ductile fracture [5]. Smith et al. [35] efficiently simulated the fracture of fragile objects under external forces. Norton et al. [36] modeled solid objects breaking due to large deformations using a mass-spring system, while Hirota et al. [37] modeled the drying process of cracked mud. Most of these methods use pre-fractured mesh forms, and when dynamic remeshing is employed, the computational load becomes too large for real-time applications.

### III. PROPOSED FRAMEWORK

#### A. TEARING CLOTH

##### 1) TRUNCATED INFORMATION OF ELEMENTS

The symbols used in this paper are shown in Table 1. In the proposed algorithm, to efficiently solve the remeshing problem, we impose a constraint that no more than two CutPoints (denoted as  $C_p$ ) can be generated on any surface or edge. This ensures that the number of CutEdges (denoted as  $C_e$ ) is minimized.  $C_e$  represents the lines created by the cutting boundary and  $C_p$  represents the points where these  $C_e$  intersect. If there are more than two  $C_p$  on a surface or edge, the average position of the intersection points is set to  $C_p$ . The information about  $C_e$  generated on the faces and edges, denoted as CutInfo or  $C_{info}$ , is represented as 6-bit data for the faces and 2-bit data for the edges.

$E_i$  represents an edge on a face, with  $E_1$  indicating the first edge.  $N_i$  denotes a node that forms a face, with  $N_1$  indicating the first node.  $C_p$  is the position of the newly created node due to the tearing of the cloth. As shown in Figure 3, the surface is divided into several patterns depending on how  $C_p$  is generated, and this information is contained in  $C_{info}$ .

Each surface is composed of 3 nodes and 3 edges, with the  $n$ -th node and the  $n$ -th edge facing each other. In  $C_{info}$  of a face, 3 bits are used to represent  $C_e$  between edges and nodes, while the remaining 3 bits are used to represent the  $C_e$  between edges (see Figure 3a). Each edge has CutInfo only if it is a CutEdge. When an edge has  $C_p$ , if  $C_e$  extends from node 0 to  $C_p$ ,  $C_{info}$  is 01<sub>(2)</sub>, and if it extends from node 1 to  $C_p$ ,  $C_{info}$  is 10<sub>(2)</sub>. If an edge does not have a CutPoint, the

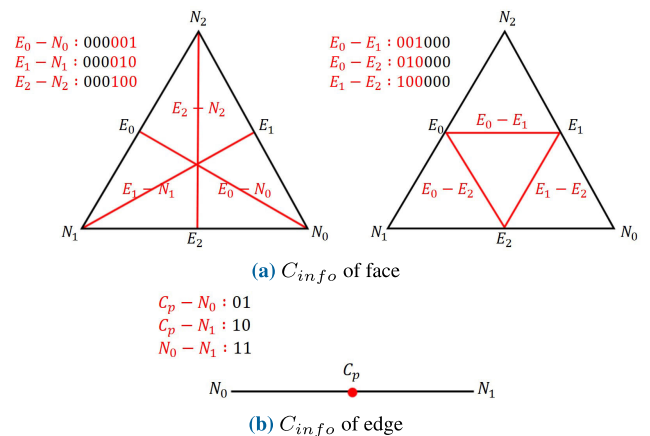
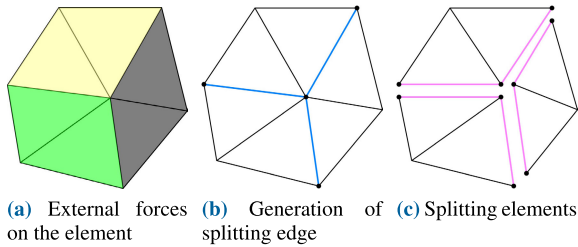


FIGURE 3.  $C_{info}$  of basic primitives ( $E$  : edge,  $N$  : node).





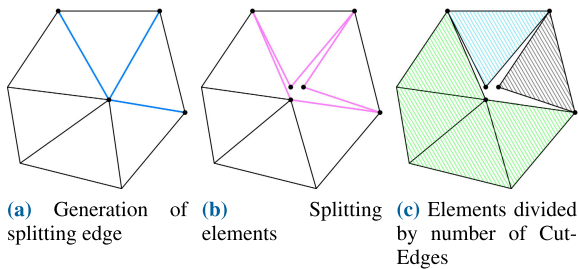
**FIGURE 4.** Creation of new nodes from a single node (color in (a) : different forces applied to each element).

edge itself becomes  $C_e$ , and its type is  $11_{(2)}$  (see Figure 3b). However, if there is only one triangle adjacent to the edge, it is uniformly set to 0.

## 2) CREATION OF NEW NODES

In this section, we explain the process of creating new node information. New nodes are generated using CutInfo, and the number of nodes is calculated as follows. For nodes, it is the number of CutEdges around the node (see Figure 4); for edges or faces, it is calculated only when there is a CutPoint. The number of newly created nodes is the number of CutEdges adjacent to the CutPoint (see Figures 5 and 6).

As shown in Figure 4, when there are 3 CutEdges in a node, the node is split into 3 nodes. It is important to ensure that the original node is preserved, so at least one new node must be created.



**FIGURE 5.** Creation of new nodes at an edge.

As shown in Figure 5, when there are 3 CutEdges on an edge, one node is split into 3 nodes. Similarly, when there are 5 CutEdges on a face, it is split into 5 nodes (see Figure 6). Consequently, new nodes are created equal to the number of CutEdges.

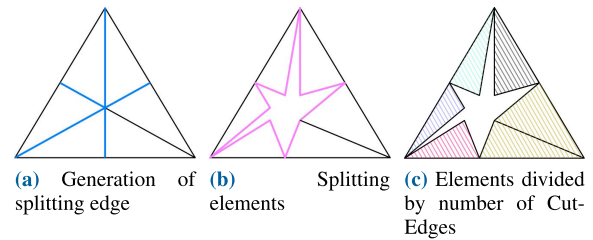
There is a point to be cautious about in the process of splitting nodes and edges. When they are not entirely surrounded by triangles, it is assumed that there is one more CutEdge around them for processing (see Figure 7).

When creating new nodes in each element, the node indices are stored using the Prefix Sum algorithm. The total number of nodes created is used to create a new node array, and the node information is stored in the corresponding node index of each element (see Figure 8). Figure 8 shows the process of storing new elements additionally created from existing elements. Then, the index to be generated in the element is

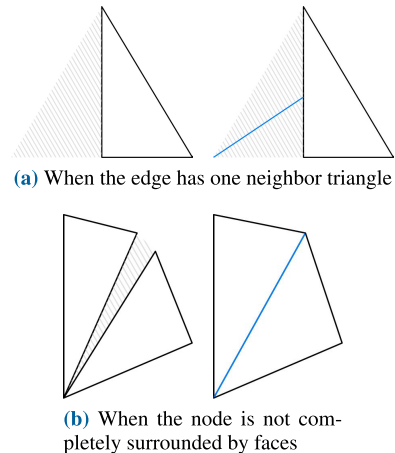
obtained through the Prefix Sum algorithm. Consequently, the new data is mapped to the index of each element.

## 3) CREATION OF NEW FACES

In this section, we explain the process of creating new information about the face. We calculate this in two cases: 1) when there is a CutPoint on the surface (see Figure 9), and 2) when there is no CutPoint on the surface (see Figure 10). If there is a CutPoint on the face, it is generally divided into 3 faces (see Figure 9b). Additionally, if there is a CutPoint on an edge, 2 more faces are further divided (see Figure 9c). In this process, the surface is divided into a minimum of 3 and up to 6 faces.



**FIGURE 6.** Creation of new nodes at face.

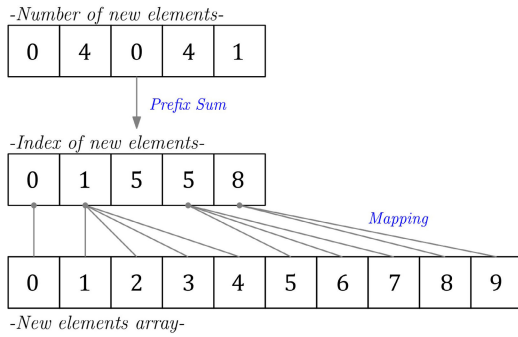


**FIGURE 7.** Case of splitting edges and nodes.

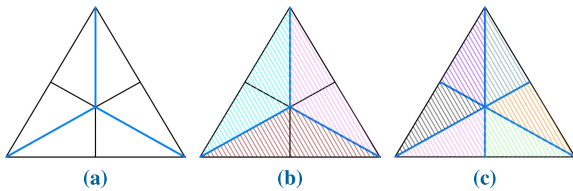
When there is no CutPoint on the face, there are three additional cases to consider: 1) If there is a CutEdge at the Edge-Node, it must be calculated first (see Figure 10). In this situation, the surface is divided primarily into 2 faces (see Figure 10b), and if there is a CutEdge on the existing edge, 2 more faces are further divided (see Figure 10c). In this process, the surface is divided into a minimum of 2 and up to 4 faces.

2) If there is one CutEdge between two edges and there is no CutPoint on the opposite edge of the CutEdge, the case is shown in Figure 11. In this scenario, the surface is divided into a total of 3 faces (see Figure 11b).

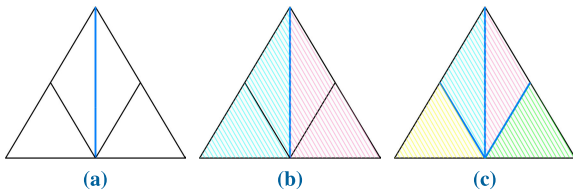
3) When there are two CutEdges between edges (see Figure 12), or when there is one CutEdge between edges with a CutPoint on the opposite edge of the CutEdge, the surface is divided into 4 faces (see Figure 13). As in Section III-A2,



**FIGURE 8.** The process of storing the numbers of new elements generated from existing elements.

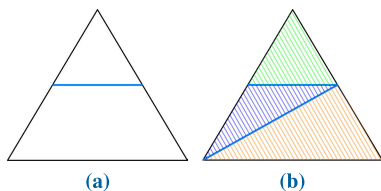


**FIGURE 9.** When a surface has a CutPoint.

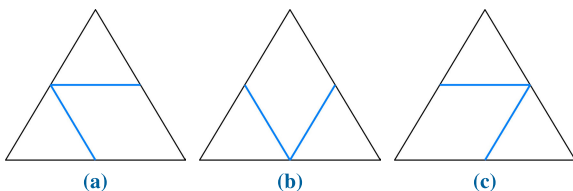


**FIGURE 10.** When a surface does not have a CutPoint and an edge-node has a CutEdge.

a new surface array is created based on the indices of the new faces to be generated for each surface and the total number of faces created (see Figure 8).



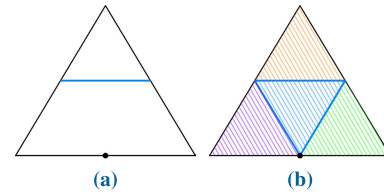
**FIGURE 11.** Edge-edge : single CutPoint.



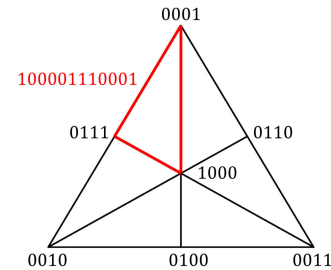
**FIGURE 12.** Edge-edge : two CutEdges.

#### 4) FACE AND NODE INFORMATION

The number of new triangles is calculated in parallel for each surface and a surface array is created with the total number of new faces calculated. The three node indices of the triangles are mapped to the created surface array. The



**FIGURE 13.** Edge-edge : single CutEdge (black dot in (a) :  $C_p$ ).



**FIGURE 14.** FaceInfo (red : new surface information, black : node information).

information required for mapping is called FaceInfo. This FaceInfo contains the positional information of the three nodes of the new face. Each node data is composed of 4 bits and FaceInfo consists of a total of 12 bits of data (see Figure 14).

The nodes need to be mapped to the new surface array using the created FaceInfo, with parallel mapping performed for the nodes, edges, and faces. For faces and edges, mapping is centered around the CutPoint, while for nodes, mapping is done by assigning the index of the newly created node to the corresponding index in the surrounding FaceInfo, centered on the node itself. When moving to the next FaceInfo, if the shared edge is a CutEdge, the node index is updated. This process is illustrated in the transition from node1 to node2 in Figure 15, and it applies to all nodes except node4. If the edge shared by the faces is not a CutEdge, the index is maintained during the mapping, which corresponds to node4 in Figure 15.

## B. SELF-COLLISION HANDLING

### 1) BVH TREE CONSTRUCTION ON GPU

To optimize BVH in a GPU environment where the goal is to improve the performance of parallel computations with limited resources, the following features should be considered: First, maximize the parallelism of the algorithm and second, minimize memory usage.

In tree update operations or tree traversal algorithms such as DFS (Depth-First Search), nodes at different levels must be calculated in sequence. In contrast, BFS (Breadth-First Search) allows nodes at the same level to be computed in parallel. To maximize the use of this characteristic, in this study, nodes are stored breadth-first, grouping nodes at the same level together. Assuming that the tree is a complete binary tree, if the current level is  $l$ , the index where the level changes is  $2^l - 1$ . Furthermore, if the index of the current

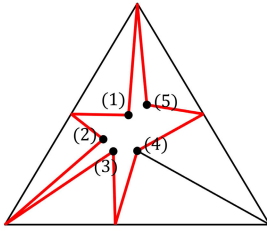


FIGURE 15. Node mapping.

node is  $n$ , the indices of the child nodes are  $2n + 1$  and  $2n + 2$ . If we are not dealing with a complete binary tree, additional data must be stored for the indices where node levels change, as well as for the indices of child and parent nodes. Therefore, to minimize memory usage more efficiently, it is best to structure the BVH tree as a complete binary tree.

Generally, a BVH-based tree requires storing information about parent and child nodes for each node. However, if the tree is structured as a complete binary tree, it can be calculated as shown in Figure 16, resulting in memory savings.

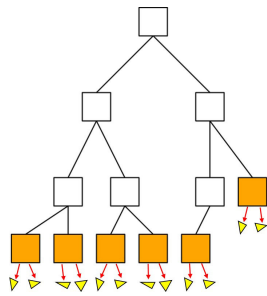


FIGURE 16. BVH tree with a complete binary tree structure (orange node : leaf node).

When performing tree traversal or updates, nodes at the same level can be computed in parallel. To handle memory access more efficiently, these nodes are stored using BFS, where nodes at the same level are stored adjacently. As mentioned above, the index of the nodes from a parent to a child node is  $2n + 1$ ,  $2n + 2$ , and from a child to a parent node, the index is  $\frac{(n-1)}{2}$ . The number of leaf nodes and the number of triangles have the following relationship:  $n_{leaf} \geq N - |\Omega_{tri}|$ , where  $N$  is the number of nodes in the BVH tree and  $\Omega_{tri}$  is the surface buffer of the triangular mesh that makes up the surfaces of the cloth.

The shape of the complete binary tree is determined by the number of triangles, and based on this shape, the number of triangles distributed during node subdivision is also determined. The number is calculated by Equation 1.

$$\begin{cases} k - 2^{L'-2} & k < \frac{3}{4}2^L \\ 2^{L'-1} & k \geq \frac{3}{4}2^L \end{cases} \quad (1)$$

Let  $L$  be the maximum level of the tree,  $l$  be the current level being subdivided, and  $L - l$  be denoted as  $L'$ . Let

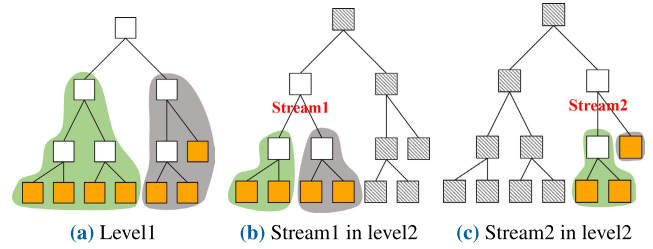


FIGURE 17. Subdivision of BVH tree node (diagonal line pattern : invalid node, orange node : leaf node, gray region : right node, green region : left node).

$k$  be the number of leaf nodes (triangles). The above equation calculates the number of triangles that need to be divided to one side. By repeating this process to divide the triangles, they are split into multiple groups. At this point, CUDA Streams are used to accelerate the parallel computation.

Taking the situation in Figure 17 as an example, in Figure 17a,  $k$  is 7, which is the same as the number of leaf nodes. The maximum level  $L$  is 3, starting with the root node at level 0. When this situation is calculated using Equation 1,  $k$  becomes  $k \geq \frac{3}{4}2^L$ . Therefore, for level 1 ( $l = 0, k' = 7$ ), it is calculated according to the lower part of Equation 1: (left)  $\leftarrow 2^{L'-1} = 4$ , : (right)  $\leftarrow k' - \text{left} = 3$ , where  $k'$  represents the number of triangles in the current node. In fact, in Figure 17a, there are 4 left nodes and 3 right nodes. The same method is applied in the next step. Since there are multiple nodes at the same level, CUDA Stream is used to perform parallel calculations. Therefore, for level 2 ( $l = 1$ ), it is calculated according to the lower part of Equation 1. For Stream 1 ( $k' = 4$ ), the nodes are as follows: (left)  $\leftarrow 2^{L'-1} = 2$ , : (right)  $\leftarrow k' - \text{left} = 2$ . As shown in Figure 17b, there are 2 left nodes and 2 right nodes. For Stream 2 ( $k' = 3$ ), the nodes are as follows: (left)  $\leftarrow 2^{L'-1} = 2$ , : (right)  $\leftarrow k' - \text{left} = 1$ . In Figure 17c, there are indeed 2 left nodes and 1 right node.

The overall process of tree construction is as follows. In a BVH tree, the number of leaf nodes is equal to the number of triangles, and since it is built in the form of a complete binary tree, the structure of the tree can be determined by knowing the number of triangles. As shown in Figure 18, the process is divided into sorting the triangles based on the position of their centroids and placing the sorted triangles in the leaf nodes of the tree. The process of sorting the triangles based on their centroid positions is further divided into four CUDA kernel functions.

First, the AABB (Axis-Aligned Bounding Box) is determined based on the centroids of the triangles. The longest axis is then selected as the splitting axis. The triangles are sorted according to the selected splitting axis and divided according to the structure of the tree. Various methods can be used to determine the splitting criteria, and we used the method proposed by Popov et al. [49]. To sort the triangles, we employed Bitonic sort, which is known to be efficient in GPU environments.

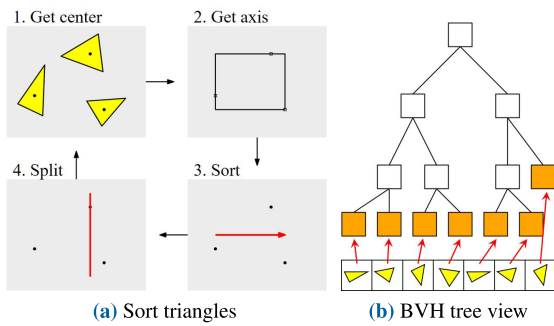


FIGURE 18. An overview of GPU-based BVH tree construction.

## 2) BVH TREE UPDATE AND TRAVERSAL ON THE GPU

In a BVH tree, the size of the AABB is updated in the order from the leaf nodes to the root node. Therefore, we design the node updates at each level to be calculated in parallel, making it efficient on a GPU architecture (see Figure 19). Since no additional memory allocation is needed and the calculations are performed in parallel, this approach can achieve high efficiency even with a small number of triangles.

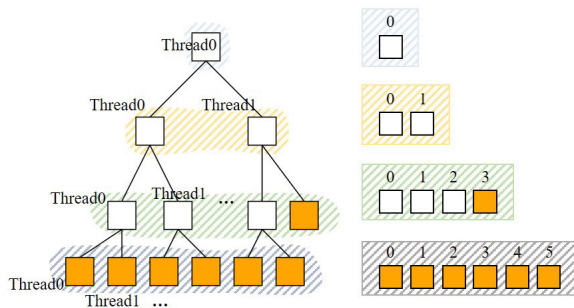


FIGURE 19. BVH tree update on the GPU (number : thread index).

For tree traversal, BFS can be used to calculate the same nodes in parallel, as in the update process. However, it requires storing data for the nodes that need to be traversed at the next level. This process often leads to exceeding memory limits in a GPU environment, especially depending on the complexity of the scene. Therefore, in this paper, we propose a CUDA-based DFS method for traversal that is not constrained by memory. The process of implementing DFS on the GPU is as follows. First, the leaf nodes are placed according to the position of each thread (see Figure 20).

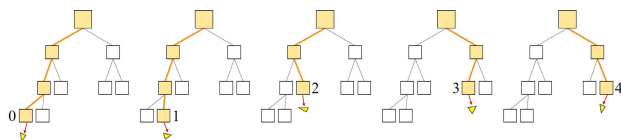


FIGURE 20. BVH tree traversal using GPU-based DFS (number : thread index).

Then, the BVH tree is traversed for the triangles contained in the leaf nodes, and collision detection is performed.

Since nodes in nearby locations are placed adjacently, they tend to follow similar traversal paths. This results in better performance in parallel computations. In summary, by placing triangles according to the position of the threads, triangles in nearby locations are stored in the same warp because of the characteristics of the BVH tree. As a result, they traverse nearly identical paths, which can lead to better parallel performance.

## 3) REPRESENTATIVE-TRIANGLE

Generally, a single vertex shares multiple faces and a single edge shares multiple faces. As a result, collision detection for vertices and edges may be performed unnecessarily multiple times. In this study, we addressed this issue using the R-Triangle (Representative-Triangle) method [50], and this process was also designed to operate within the GPU kernel.

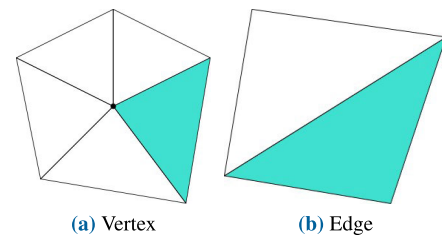


FIGURE 21. Collision candidates collected through the R-Triangle (cyan color : smallest index).

In the R-Triangle method, the representative triangle for a vertex and edge is efficiently determined. This method performs collision detection for a vertex only in the representative triangle among the triangles that share the vertex. Although there are various ways to select the representative triangle, in this study we chose the triangle with the smallest index among the shared triangles as the representative triangle to ensure efficient processing (see Figure 21).

## 4) EXPERIMENTAL SCENARIO

In this section, we examine an example of constructing a BVH based on our method. First, the triangles that make up the cloth surfaces are hierarchically sorted based on their indices, with parallelization performed for each segment using CUDA Streams. This process is repeated until each triangle becomes a leaf node (see Figure 22).

If the cloth deforms and the positions of the triangle vertices ( $p_0, p_1, p_2$ ) change, the BVH constructed as a complete binary tree updates the size of the AABB in the order from the leaf nodes to the root node (see Figure 23).

Figure 24 visualizes the GPU-based BVH tree using the complete binary tree proposed in this study. Although BFS is easier to parallelize, it requires more memory as the tree size increases. Therefore, in this paper, we address this issue by using GPU-based DFS. To implement DFS on the GPU, the triangles corresponding to the leaf nodes are assigned to the threads according to their positions and BVH traversal is



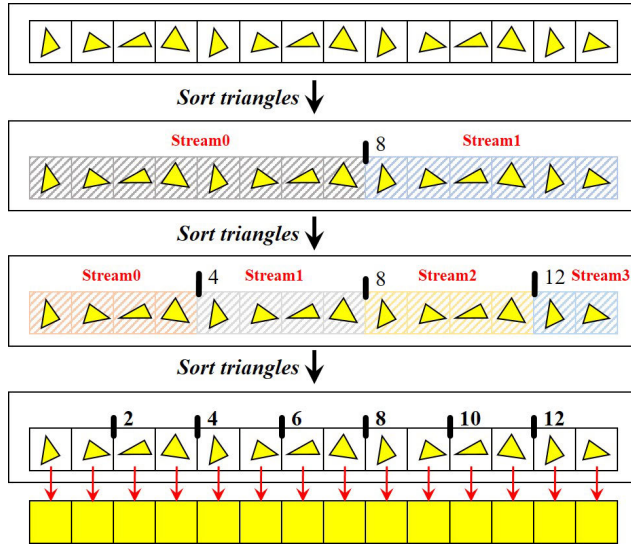


FIGURE 22. BVH construction (yellow node : leaf node).

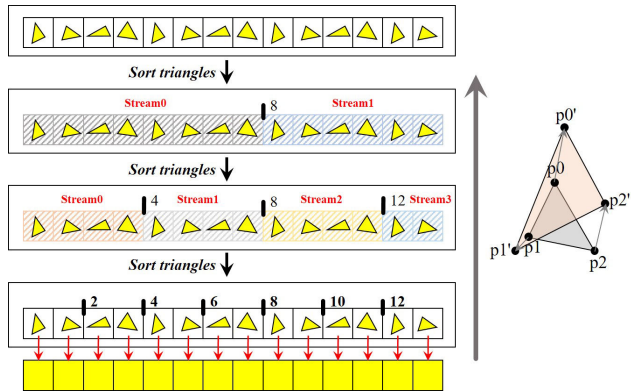


FIGURE 23. Refitting BVH (gray arrow : update direction,  $pos^t : (p_0, p_1, p_2), pos^{t+\Delta t} : (p'_0, p'_1, p'_2)$ ).

performed on these triangles to conduct collision detection (see Figure 25).

#### IV. EXPERIMENTAL RESULTS

To produce the results of this study, we used a computer equipped with an AMD Ryzen Threadripper PRO 3955WX 16-Cores CPU, 256GB RAM, and a Geforce RTX 3090 GPU. For the cloth simulation, we utilized the Jacobian Projective Dynamics algorithm, which is computationally efficient even when constraints change and is friendly to parallel programming. Furthermore, we accelerated the computation using the Chebyshev iterative method proposed by Wang [42]. The Prefix Sum algorithm was computed using the exclusive-scan function from the CUDA Thrust library. During the cloth-tearing process, the triangle-based dynamic remeshing patterns proposed in this study were passed to CUDA threads for parallel computation. The self-collision handling in the tearing cloth simulation was optimized on the GPU using a complete binary tree-based approach.

#### A. CLOTH TEARING SIMULATION

Figure 26 shows a single frame from a real-time simulation of cloth being cut in six directions at different resolutions. The cloth is not cut according to the original shape of the triangles, but rather in a natural manner that matches the direction and shape of the cut. In most real-time applications, elements are typically cut along the edges of triangles to ensure performance. However, our method dynamically handled remeshing in real-time, regardless of the cloth's resolution.

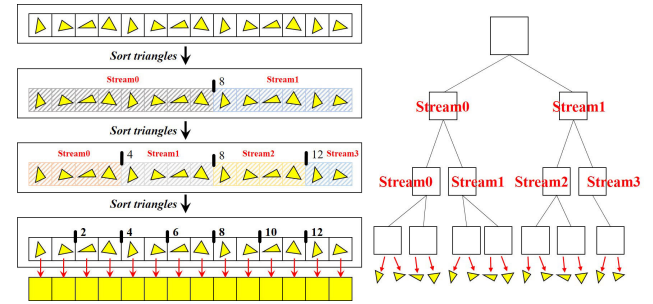


FIGURE 24. Visualization of BVH tree.

Figure 27 shows a scene where the user is cutting the cloth in real-time in arbitrary directions, rather than in predetermined directions. The cut is defined in a straight line by the mouse drag motion, and accordingly, dynamic remeshing and cloth simulation are handled in real-time.

Figure 28 shows a scene in which the cloth is torn from a position selected by the user with a mouse. Here, the cloth is designed to tear when the external force applied exceeds a threshold. Since the proposed method uses GPU-based dynamic remeshing, it accurately represents detailed tearing patterns in real-time, regardless of cloth resolution. When comparing high resolution and low resolution, there was no significant difference in the surface details expressed after tearing.

Figure 31 shows the results of an experiment conducted on an object with a 3D curved surface, rather than a planar cloth. As with the previous results, the cloth was cut in random patterns, and the figure effectively demonstrates that dynamic remeshing was performed in real-time. Despite the higher number of cuts compared to the previous scenes, the surface was successfully torn in real-time.

Figure 32 shows the results of a cutting simulation performed on a model with a high polygon count. As with the previous results, dynamic remeshing was successfully performed in real-time, even on surfaces composed of a large number of triangles.

Figure 33 shows the result of a scene in which the cloth shrinks or melts due to heat. Unlike the previous results, this model simulates the cloth melting or shrinking when heat diffusion progresses through the triangular mesh and the heat exceeds a certain threshold. This scene is the result of integrating the method proposed by Jeong et al. [43] with our method. As shown in the previous results, the tearing simulation of the cloth performed well in real-time in all

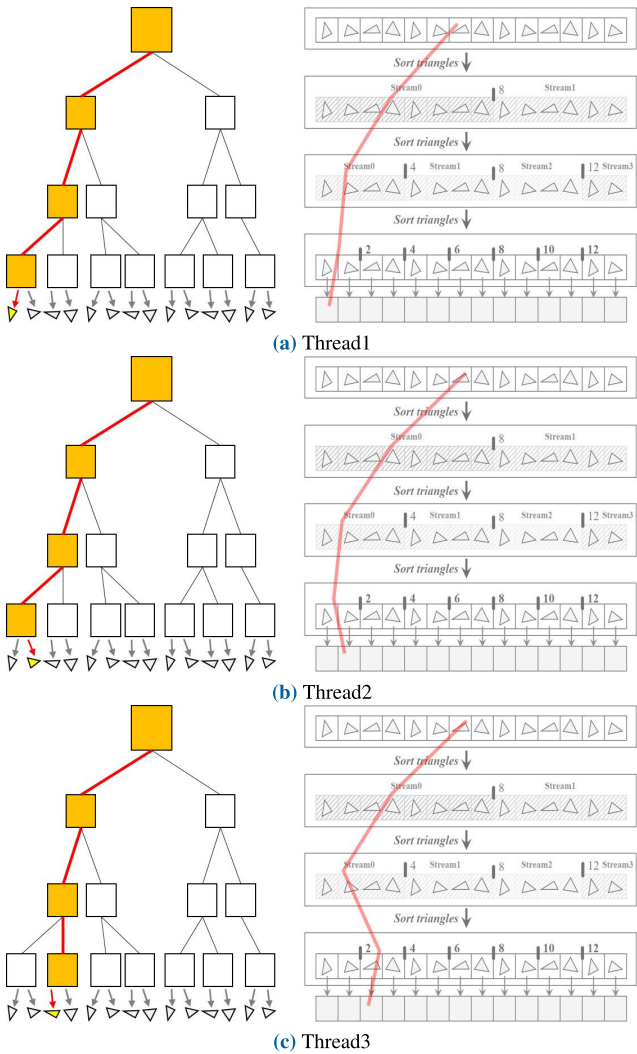


FIGURE 25. BVH tree traversal on GPU kernel.

cases. In the figure, the red dot indicates the location where heat is applied, and it was made controllable intuitively via mouse input. This scene was tested on both low- and high-resolution surfaces, and there was no significant difference in detail due to resolution during the tearing simulation process, with all operations running in real-time.

B. SELF-COLLISION HANDLING IN CLOTH SIMULATION

Figure 29 shows the results of the self-collision handling based on the GPU-based BVH tree proposed in this paper. The cloth was represented at various resolutions and a scene was constructed to induce frequent self-collisions. The same scene was then processed using the existing CPU-based BVH tree to compare performance differences, with the results recorded in Table 2.

Table 2 records the construction time, update time and traversal time of the BVH tree. When the number of triangles on the cloth surfaces is less than 30,000, the GPU is slower than the CPU due to memory allocation time. However, when

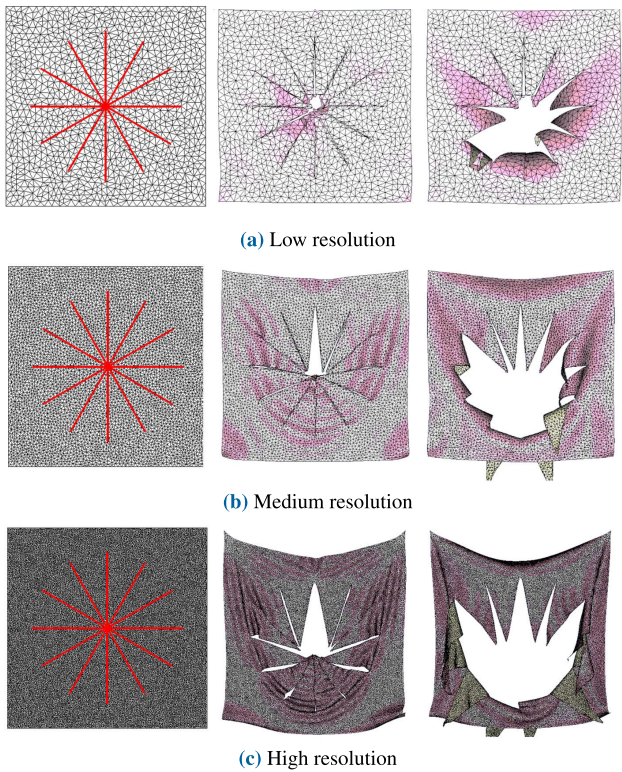


FIGURE 26. Scene of cloth tearing in six directions.

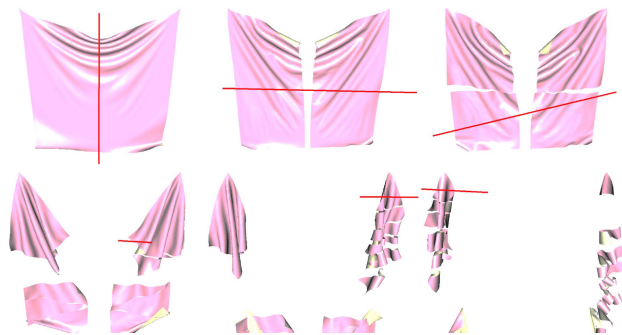


FIGURE 27. Cloth being cut in real-time according to user interaction.

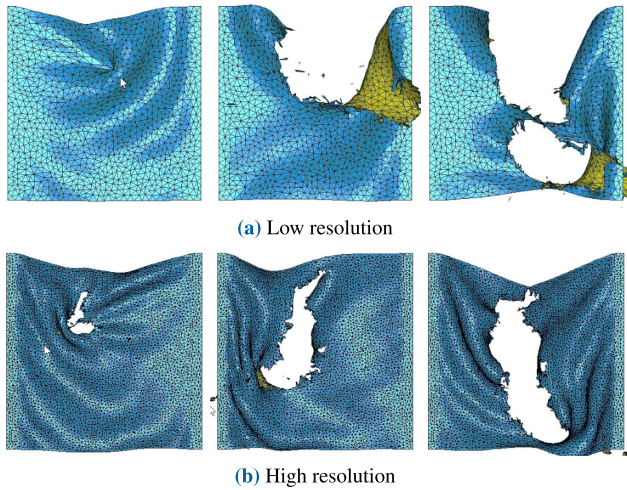
the number exceeds 30,000, the speed improves about 5 to 10 times.

TABLE 2. Comparison of time spent on BVH tree construction, update (refit), and traversal between GPU and CPU.

| # of Tri. | Construction<br>GPU, CPU | Update(refit)<br>GPU, CPU | Traversal<br>GPU, CPU |
|-----------|--------------------------|---------------------------|-----------------------|
| 2,244     | 23.94, 5.71ms            | 0.16, 0.31 $\mu$ s        | 1.19, 10.28ms         |
| 27,030    | 59.12, 45.18ms           | 0.18, 1.28 $\mu$ s        | 12.52, 162.92ms       |
| 35,992    | 0.12, 0.98s              | 0.35, 4.81 $\mu$ s        | 0.41, 8.65s           |

The BVH tree update does not require additional memory allocation and involves only running kernel functions. Moreover, since it is calculated in parallel, it shows good performance, with speeds up to 10 times faster than the

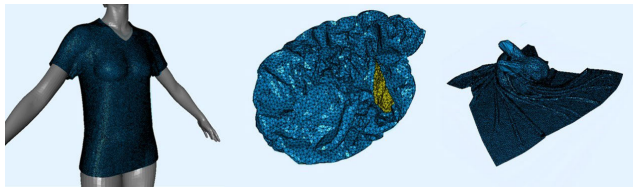




**FIGURE 28.** Real-time cloth tearing with user interaction.



**FIGURE 29.** Self-collision handling with our method (left to right : low→high resolution).



**FIGURE 30.** Various collision scenes with our method.

CPU. Although the traversal time of the BVH tree is less parallelized because it uses DFS instead of BFS, it still shows a significant improvement, with speeds increasing by 10 to 20 times.

Figure 30 shows the results of the experiment with our method in a wider variety of scenes. In addition to the self-collision handling shown earlier, cloth-object collision handling was also performed stably.

## V. DISCUSSION

In this section, we analyze the computational performance of our method by implementing the scene shown earlier using various approaches for comparison. This paper focuses solely on tearing, not on cloth dynamics, and the key method required in this process is remeshing. Therefore, we compare the performance of 1) remeshing methods for fast tearing simulation, 2) handling dynamic remeshing, and 3) GPU-based dynamic meshing using our method.

Table 3 compares the performance of each scene, including not only tearing but also dynamics and collision handling.

As shown in Table, the GPU-based method proposed in this paper demonstrated significantly faster results compared to the CPU-based method and the previous method [3]. In particular, in the previous method, the tearing is performed according to the resolution of the input mesh, rather than through dynamic remeshing, which results in slightly lower tearing quality on front surfaces compared to our method. However, because of its faster speed, it was compared with our method. However, since the previous method operates on the CPU, our GPU-focused method showed significantly faster results in terms of computational speed and was capable of real-time processing, making it highly suitable for real-time applications like virtual reality or the Metaverse. Additionally, as the number of tearing patterns or cutting planes increased, the computational load increased for both the previous method and our method (CPU). However, the difference was not significant in our method (GPU), allowing real-time cloth tearing even at high resolution.

Table 4 compares the performance of the tearing simulation alone. Since our method's dynamic remeshing is optimized for the GPU, the number of cutting planes or the resolution of the cloth had only a slight effect on performance, without significantly impacting it. The computational time was compared using the min / max / mean values, while Figure 26 excludes the min / max performance as it only considers predefined cutting planes. However, our method demonstrated sufficiently fast performance in all cases, enabling real-time processing.

The pseudocode summarizing the GPU-accelerated dynamic remeshing algorithm is shown in Table 5. The size of  $bs$  used when calling the CUDA kernel function is set to 1024, and the cutting planes defined in line2 are predefined shapes, as shown in Figure 26, or real-time shapes defined by the user through mouse drag motions. line3 is the process of calculating whether any elements are being cut by the cutting planes. Information related to the cut,  $C_{info}$ , is collected and this information is used to proceed with the element splitting process proposed in this paper. In line4, new nodes are created based on the cutting, and if the number of new nodes is not equal to the number of original cloth nodes, this means that the node has been split, and new faces are created using the method proposed in this paper (see line6~line8). Detailed descriptions of each CUDA kernel are provided in line11~line22.  $bs'$  is the refined block size, and is set differently for each CUDA kernel.

## A. COMPARISON WITH PREVIOUS METHODS

### 1) DYNAMIC REMESHING

In this section, we explain the differences between our method and previous methods for representing cutting surfaces in deformable bodies.

In the field of physics-based simulation, modeling fracture and tearing simulation involves two main processes: 1) dynamic remeshing to cut the elements and nodes of the mesh due to applied forces, and 2) calculating elasticity forces from the cut elements. This paper proposes a method

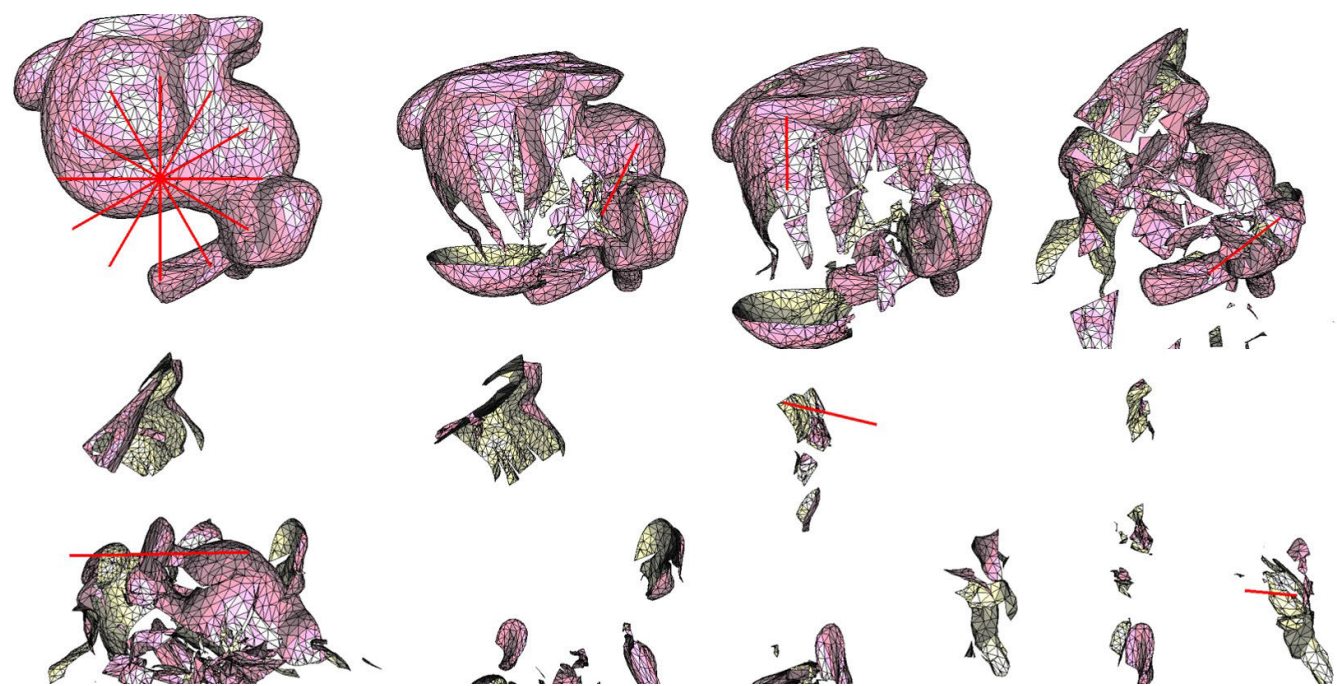


FIGURE 31. Real-time tearing of the Stanford Bunny by user interaction.

TABLE 3. Scene configuration and performance (\* : our method(GPU), \* : our method(CPU), † : previous method).

| Scene | Frames | # of faces | # of vertices | Computational mean time : ours(GPU), ours(CPU), previous method [3] | Performance : * vs. *, * vs. † |
|-------|--------|------------|---------------|---------------------------------------------------------------------|--------------------------------|
| 26a   | 115    | 123        | 2,244         | 4.3ms, 3.1s, 3.0s                                                   | ×720, ×697                     |
| 26b   | 115    | 2,244      | 6,732         | 9.1ms, 9.4s, 9.2s                                                   | ×1,032, ×1,010                 |
| 26c   | 115    | 35,992     | 107,976       | 9.1ms, 17.5s, 16.4s                                                 | ×1,923, ×1,802                 |
| 27    | 95     | 35,992     | 107,976       | 10.1ms, 23.8s, 22.0s                                                | ×2,356, ×2,178                 |
| 28a   | 102    | 123        | 2,244         | 4.3ms, 4.3s, 3.9s                                                   | ×1,000, ×906                   |
| 28b   | 102    | 2,244      | 6,732         | 9.1ms, 11.9s, 11.3s                                                 | ×1,307, ×1,241                 |
| 33a   | 102    | 123        | 2,244         | 4.3ms, 3.4s, 3.1s                                                   | ×790, ×720                     |
| 33b   | 102    | 2,244      | 6,732         | 9.1ms, 10.3s, 10.1s                                                 | ×1,131, ×1,109                 |

TABLE 4. Scene configuration and performance for only tearing.

| Scene | Frames | # of faces | # of vertices | (Only tearing) Computational time : min, max, mean | # of cutting planes |
|-------|--------|------------|---------------|----------------------------------------------------|---------------------|
| 26a   | 115    | 2,244      | 6,732         | —ns, —ns, 50,615ns                                 | 6                   |
| 26b   | 115    | 9,010      | 27,030        | —ns, —ns, 62,000ns                                 | 6                   |
| 26c   | 115    | 35,992     | 107,976       | —ns, —ns, 95,272ns                                 | 6                   |
| 31    | 92     | 4,968      | 14,904        | 42,735ns, 74,277ns, 52,994ns                       | 91                  |
| 32    | 84     | 100,000    | 300,000       | 155,782ns, 230,020ns, 189,527ns                    | 82                  |

for optimizing the dynamic remeshing process on the GPU. There are generally six methods for cutting surface meshes and tetrahedral meshes: 1) Element deletion [51], 2) Splitting along existing element faces [8], [17], 3) Element duplication [18], [19], 4) Snapping of vertices [20], 5) Element refinement [6], [9], [21], [22], [23], and 6) Combined snapping of vertices and element refinement [24].

The methods of element deletion and splitting along existing element faces cut along the elements of the original discretization but often result in jagged surfaces,

creating sharp edges. Using snapping of vertices and element refinement methods, or a combination of both, allows for precise representation of cut surfaces. However, as elements are modified, these approaches require measures to prevent ill-shaped elements. The element duplication method enables accurate and robust fracture simulation by including accurate surfaces in the duplicated elements with their original shapes. However, the duplication process consumes significant memory and computation time, making it challenging to use in real-time applications.



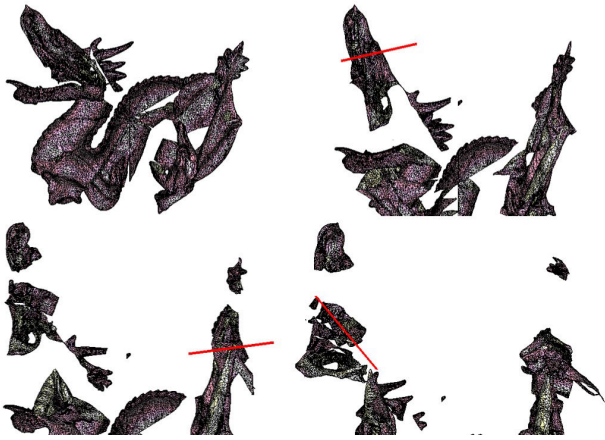


FIGURE 32. Real-time tearing of the Stanford Dragon by user interaction.

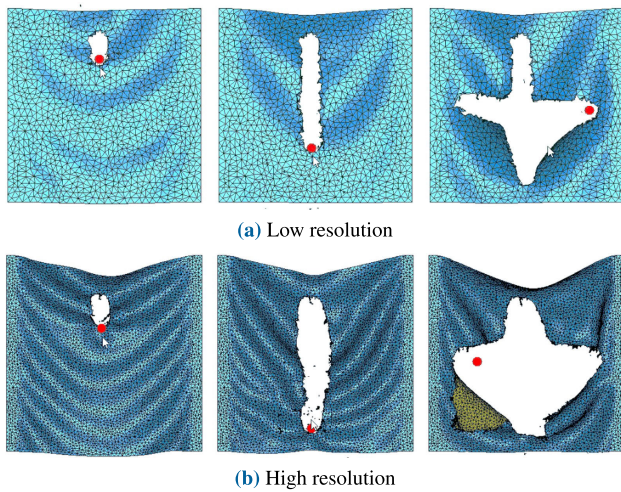


FIGURE 33. Real-time cloth tearing by heat diffusion.

Some studies have used spatial data structures to optimize and reduce the computational load of surface calculations. Certain techniques create polygonal surface meshes through the voxelization method, which directly represents cut surfaces without ill-shaped elements. These techniques employ regular grids or adaptive grid meshes, such as octrees, to represent cutting. However, since they do not dynamically and directly remesh the original mesh, their accuracy in representing cutting planes is lower, allowing only for large-scale cuts. Thus, they are not capable of representing the complex cutting planes or fragmented element shapes demonstrated in this study.

In dynamic remeshing, it is generally important to perform cuts while stably calculating elasticity forces, making it crucial to avoid ill-shaped elements when splitting elements. However, since the shape and connectivity information of the mesh are constantly changing, a significant amount of computation time and memory usage is required for updates. This paper is the first study in the field of physics-based simulation to optimize dynamic remeshing in a GPU-friendly manner. More specifically, it can be described as a study

TABLE 5. Pseudocode for the GPU-accelerated dynamic remeshing algorithm.

```

 $n_{num}$  : number of nodes
 $f_{num}$  : number of faces
 $e_{num}$  : number of edges
 $bs$  : block size
 $bs'(x, y) \leftarrow \max\left(\frac{x+y-1}{y}, 1u\right)$ 
1: procedure: simulation()
2:   Predefined or real-time cutting planes;
3:   If(getElementCutInfo()) {
4:     generateNewNode();
5:     If(news.size !=  $n_{num}$ ) {
6:       generateNewFaces();
7:       buildNewAdjacency();
8:       computeNewNodeParams();
9:     }
10:  }
11: procedure: getElementCutInfo()
12:   getElementsCutInfoKernel<<<bs'(fnum, bs), bs>>>>();
13:   getPointWeightEdgeKereneel<<<bs'(enum, bs), bs>>>>();
14:   getPointWeightFaceKernel<<<bs'(nnum, bs), bs>>>>();
15: procedure: generateNewNode()
16:   numberCutEdgesFaceKernel<<<bs'(fnum, bs), bs>>>>();
17:   numberCutEdgesEdgeKernel<<<bs'(enum, bs), bs>>>>();
18:   numberCutEdgesNodeKernel<<<bs'(nnum, bs), bs>>>>();
19:   numberNewNodesFaceKernel<<<bs'(fnum, bs), bs>>>>();
20:   numberNewNodesEdgeKernel<<<bs'(enum, bs), bs>>>>();
21:   numberNewNodesNodeKernel<<<bs'(nnum, bs), bs>>>>();
22:   generateNewNodesNodeKernel<<<bs'(nnum, bs), bs>>>>();
23:   generateNewNodesEdgeKernel<<<bs'(enum, bs), bs>>>>();
24:   generateNewNodesFaceKernel<<<bs'(fnum, bs), bs>>>>();
25: procedure: generateNewFaces()
26:   numberNewFaceInfoKernel<<<bs'(fnum, bs), bs>>>>();
27:   computeNewFaceInfoKernel<<<bs'(fnum, bs), bs>>>>();
28:   newFaceMappingEdgeKernel<<<bs'(enum, bs), bs>>>>();
29:   newFaceMappingNodeKernel<<<bs'(nnum, bs), bs>>>>();

```

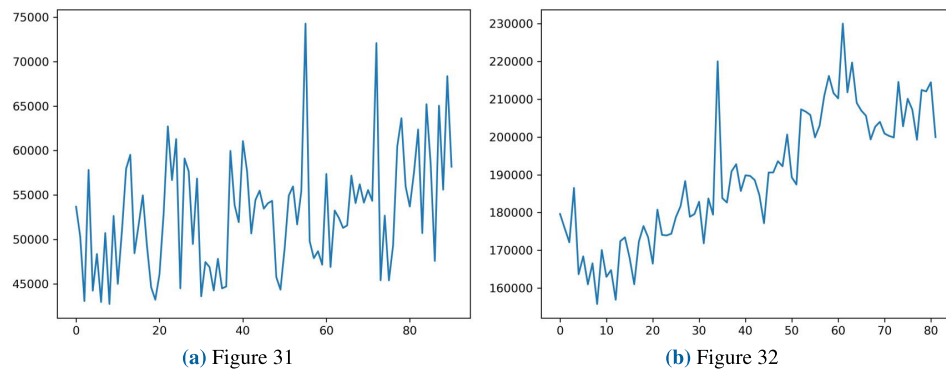
that optimizes a method combining snapping of vertices and element refinement on the GPU.

## 2) SELF-COLLISION HANDLING

In this section, we analyze the differences between our method and existing methods in the collision handling process. There is extensive research on self-collision handling for deformable bodies, with hashing and BVH trees being the most commonly used data structures. Ultimately, for collision handling, determining collisions with adjacent triangles is essential, which is why BVH trees are widely used as a data structure (more frequently than hash tables, as BVH does not require fixed spacing like a regular grid). Our method optimizes the construction, updating, and traversal of the BVH tree on the GPU. By structuring triangular meshes as a complete binary tree, our approach can be computed efficiently using GPU kernels, making it applicable to various fields that utilize BVH trees.

## B. METHOD ANALYSIS

In this section, we will analyze our method through several questions.



**FIGURE 34.** Comparison of computational time for only tearing.

### 1) REALISTIC SIMULATION OF TEAR PROPAGATION REQUIRES ROBUST STRESS AND STRAIN CALCULATIONS. HOW THE RESPECTIVE STRESS AND STRAIN CALCULATIONS ARE PERFORMED WHICH DETERMINES WHEN AND WHERE THE CLOTH SHOULD TEAR?

The method proposed in this study is not about calculating split positions or strain when cloth is torn but is a dynamic remeshing algorithm executed when vertices or edges are split. Generally, when using tetrahedra-based volumetric meshes, the strain and stress of an element are calculated, and a separation tensor is derived through a force decomposition process. Ultimately, the eigenvectors of this tensor matrix are used to cut the element. This is the physics-based approach for fracturing volumetric meshes.

However, our method does not calculate forces to fracture elements but instead focuses on handling dynamic remeshing resulting from these forces. Most of the results presented in this study are based on tearing driven by user mouse interactions rather than strain and stress tensors. Of course, as shown in Figure 28, there are results where cloth is grabbed and torn apart. In this case, the method used calculates tearing simulation based on the length of the springs connected to the triangular mesh. For detailed information on this method, please refer to the following techniques [52]

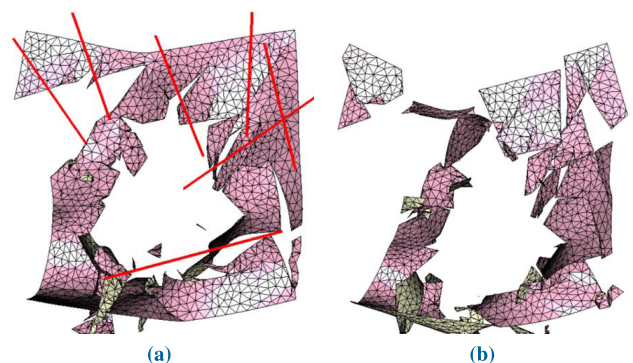
### 2) WHAT IS THE COLLISION DETECTION THROUGHPUT?

Since collision detection throughput varies depending on scene complexity, it is difficult to measure processing throughput. Therefore, in this paper, we measured the computation speed for each scene, including tearing and collision handling, as shown in Table 3. Additionally, we conducted experiments in scenes with frequent collisions rather than tearing scenarios to measure computation speed (see Figure 30 and Table 2). Moreover, the processes of cloth simulation, tearing, and collision handling are entirely handled on the GPU. As shown in Figure 31, GPU utilization was measured at 57%.

### 3) HOW IS THE MESH QUALITY PRESERVED DURING AND AFTER REMESHING?

The mesh quality after remeshing was stable without any significant ill-posed elements, and therefore, no situations

arose where dynamics could not be properly calculated due to such elements. Additionally, all scenarios tested in this study showed stable results without any unstable artifacts. When remeshing triangles, we aimed to create shapes as close to equilateral triangles as possible. In cases where this was not achievable (e.g., elements where forces could not be properly calculated due to ill-posed elements), a snapping approach was applied to prevent problems. No additional post-processing steps were added to maintain mesh quality beyond this. Moreover, the dynamic remeshing acceleration algorithm proposed in this study can be easily integrated with mesh quality maintenance techniques, making it highly flexible and broadly applicable.



**FIGURE 35.** Cloth surface remeshing by the cutting edge.

Figure 35 shows the results of applying a cutting plane using our method to split cloth surfaces and add dynamics to the newly created triangles. It effectively demonstrates how surface triangles are split according to the direction and length of the cutting plane. Additionally, sharp triangles, characteristic of dynamic remeshing near the cutting plane, were formed, and the dynamics were stably calculated without any popping or jittering.

### 4) HOW THE PROPOSED METHOD TRACKS WHETHER REMESHING CONSERVES THE STRUCTURAL INTEGRITY OF THE CLOTH?

The proposed method is a framework for performing dynamic remeshing on triangular mesh-based cloth with

GPU optimization. Since the basic calculation unit of cloth dynamics is the triangle element, it is challenging to apply this method to yarn-level cloth. Typically, specific metrics are devised to track the quality of the cloth, and after remeshing, changes in these metrics are considered to maintain the structural integrity of the cloth. However, our method does not aim to maintain structural integrity after dynamic remeshing but focuses on achieving high-speed dynamic meshing on the GPU.

Of course, the perspective you pointed out is an important issue. Our method assumes that the cloth is an elastic material. To represent this, we calculated cloth dynamics using Projective Dynamics and the Chebyshev iterative method, and these dynamics were directly applied even after remeshing. Although we did not include post-processing steps to maintain the structural integrity of the cloth after remeshing, we confirmed through visual simulations that the tearing cloth effectively expressed the characteristics of an elastic material in all results.

## VI. CONCLUSION

In this paper, we propose a new framework and data structure for efficiently handling cloth tearing simulation on the GPU. Additionally, to optimize the self-collision handling process, we introduce a GPU-based complete binary tree method to efficiently construct, update, and traverse the BVH tree, and we minimize primitive collision checks by optimizing the R-Triangle technique for triangular meshes on the GPU. Existing methods have focused on updating geometry and connectivity information to handle dynamic remeshing. This study is the first approach to extend this into a customized cloth tearing simulation algorithm for the GPU. In particular, since the mesh structure changes as the vertex and surface information is updated at each step, it posed a challenge to develop a simple structure for the GPU. However, our study effectively addressed this issue. Although the proposed method focuses on thin shells, we plan to extend it to tetrahedra-based volumetric meshes for GPU-based fracture simulation in the future. The proposed method can be applied to various interactive content in real time, including virtual reality, games, and the Metaverse, which require real-time interaction with users.

However, our research has several limitations. Since we focused on designing the complex dynamic remeshing structure needed for tearing simulation on the GPU, we were unable to apply more GPU architecture-based memory optimizations. Additionally, because our approach does not duplicate elements as in the virtual node algorithm, numerical errors can occur if snapping is not applied when cutting planes are close to elements [18]. Duplicating elements could lead to inefficient use of GPU memory, making it difficult to extend the current version of the algorithm to incorporate the virtual node algorithm. We plan to improve the algorithm in the future to address these issues on the GPU.

## REFERENCES

- [1] P. Meseure, J. Davanne, L. Hilde, J. Lenoir, L. France, F. Triquet, and C. Chaillou, "A physically-based virtual environment dedicated to surgical simulation," in *Proc. Surgery Simul. Soft Tissue Modeling, Int. Symp.*, France, Jan. 2003, pp. 38–47.
- [2] Q. Tan, Z. Pan, L. Gao, and D. Manocha, "Realtime simulation of thin-shell deformable materials using CNN-based mesh embedding," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 2325–2332, Apr. 2020.
- [3] M. S. Souza, A. Wangenheim, and E. Comunello, "Fast simulation of cloth tearing," *J. Interact. Syst.*, vol. 5, no. 1, pp. 44–48, Jul. 2014.
- [4] Q. Guo, X. Han, C. Fu, T. Gast, R. Tamstorf, and J. Teran, "A material point method for thin shells with frictional contact," *ACM Trans. Graph.*, vol. 37, no. 4, pp. 1–15, Aug. 2018.
- [5] J. F. O'Brien, A. W. Bargteil, and J. K. Hodgins, "Graphical modeling and animation of ductile fracture," in *Proc. 29th Annu. Conf. Comput. Graph. Interact. Techn.*, 2002, pp. 291–294.
- [6] D. Bielser, V. A. Maiwald, and M. H. Gross, "Interactive cuts through 3-dimensional soft tissue," *Comput. Graph. Forum*, vol. 18, no. 3, pp. 31–38, Sep. 1999.
- [7] S. Cotin, H. Delingette, and N. Ayache, "A hybrid elastic model allowing real-time cutting, deformations and force-feedback for surgery training and simulation," *Vis. Comput.*, vol. 16, no. 8, pp. 437–452, 2000.
- [8] H.-W. Nienhuys and A. F. V. D. Stappen, "Combining finite element deformation with cutting for surgery simulations," in *Proc. EUROGRAPHICS*, Jan. 2000, pp. 1–9.
- [9] A. B. Mor and T. Kanade, "Modifying soft tissue models: Progressive cutting with minimal new element creation," in *Proc. 3rd Int. Conf.*, Jan. 2000, pp. 598–607.
- [10] L. Jeřábková, G. Bousquet, S. Barbier, F. Faure, and J. Allard, "Volumetric modeling and interactive cutting of deformable bodies," *Prog. Biophys. Mol. Biol.*, vol. 103, nos. 2–3, pp. 217–224, Dec. 2010.
- [11] J. Wu, C. Dick, and R. Westermann, "Interactive high-resolution boundary surfaces for deformable bodies with changing topology," *Vriphys*, vol. 11, pp. 29–38, Jan. 2011.
- [12] M. Seiler, D. Steinemann, J. Spillmann, and M. Harders, "Robust interactive cutting based on an adaptive octree simulation mesh," *Vis. Comput.*, vol. 27, nos. 6–8, pp. 519–529, Jun. 2011.
- [13] S. Martin, P. Kaufmann, M. Botsch, M. Wicke, and M. Gross, "Polyhedral finite elements using harmonic basis functions," *Comput. Graph. Forum*, vol. 27, no. 5, pp. 1521–1529, Jul. 2008.
- [14] M. Müller, R. Keiser, A. Nealen, M. Pauly, M. Gross, and M. Alexa, "Point based animation of elastic, plastic and melting objects," in *Proc. ACM SIGGRAPH/Eurographics Symp. Comput. Animation*, 2004, pp. 141–151.
- [15] N. Pietroni, F. Ganovelli, P. Cignoni, and R. Scopigno, "Splitting cubes: A fast and robust technique for virtual cutting," *Vis. Comput.*, vol. 25, no. 3, pp. 227–239, Mar. 2009.
- [16] O. C. Zienkiewicz, R. L. Taylor, and J. Z. Zhu, *The Finite Element Method: Its Basis and Fundamentals*. Amsterdam, The Netherlands: Elsevier, 2005.
- [17] M. Müller and M. Groß, "Interactive virtual materials," *Graph. Interface*, vol. 2004, pp. 239–246, May 2004.
- [18] N. Molino, Z. Bao, and R. Fedkiw, "A virtual node algorithm for changing mesh topology during simulation," *ACM Trans. Graph.*, vol. 23, no. 3, pp. 385–392, Aug. 2004.
- [19] E. Sifakis, K. G. Der, and R. Fedkiw, "Arbitrary cutting of deformable tetrahedralized objects," in *Proc. ACM SIGGRAPH/Eurographics Symp. Comput. Animation*, Aug. 2007, pp. 73–80.
- [20] H.-W. Nienhuys and A. F. V. D. Stappen, "A surgery simulation supporting cuts and finite element deformation," in *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.*, Jan. 2001, pp. 145–152.
- [21] D. Bielser and M. H. Gross, "Interactive simulation of surgical cuts," in *Proc. 8th Pacific Conf. Comput. Graph. Appl.*, 2000, pp. 116–442.
- [22] D. Bielser, P. Ghardon, M. Teschner, and M. Gross, "A state machine for real-time cutting of tetrahedral meshes," in *Proc. 11th Pacific Conf. Comput. Graph. Appl.*, 2003, pp. 377–386.
- [23] F. Ganovelli, P. Cignoni, C. Montani, and R. Scopigno, "A multiresolution model for soft objects supporting interactive cuts and lacerations," *Comput. Graph. Forum*, vol. 19, no. 3, pp. 271–281, Sep. 2000.
- [24] D. Steinemann, M. Harders, M. Gross, and G. Székely, "Hybrid cutting of deformable solids," in *Proc. IEEE Virtual Reality Conf. (VR)*, Mar. 2006, pp. 35–42.
- [25] J. R. Shewchuk, "What is a good linear finite element? Interpolation, conditioning, anisotropy, and quality measures," Univ. California Berkeley, Berkeley, CA, USA, Tech. Rep., 2002.



- [26] Y. Wang, C. Jiang, C. Schroeder, and J. Teran, "An adaptive virtual node algorithm with robust mesh cutting," in *Proc. Symp. Comput. Animation*, Jul. 2014, pp. 77–85.
- [27] M. Desbrun and M.-P. Gascuel, "Animating soft substances with implicit surfaces," in *Proc. 22nd Annu. Conf. Comput. Graph. Interact. Techn.*, 1995, pp. 287–290.
- [28] M. Müller, R. Keiser, A. Nealen, M. Pauly, M. Gross, and M. Alexa, "Animating soft substances with implicit surfaces," in *Proc. ACM SIGGRAPH/Eurographics Symp. Comput. Animation*, 2004, pp. 141–151.
- [29] M. Pauly, R. Keiser, B. Adams, P. Dutré, M. Gross, and L. J. Guibas, "Meshless animation of fracturing solids," *ACM Trans. Graph.*, vol. 24, no. 3, pp. 957–964, Jul. 2005.
- [30] V. Spandan, D. Lohse, M. D. de Tullio, and R. Verzicco, "A fast moving least squares approximation with adaptive Lagrangian mesh refinement for large scale immersed boundary simulations," *J. Comput. Phys.*, vol. 375, pp. 228–239, Dec. 2018.
- [31] Z. Wang, P. Wang, P. Wang, Q. Dong, J. Gao, S. Chen, S. Xin, C. Tu, and W. Wang, "Neural-IMLS: Self-supervised implicit moving least-squares network for surface reconstruction," 2021, *arXiv:2109.04398*.
- [32] D. Baraff and A. Witkin, "Large steps in cloth simulation," in *Proc. 25th Annu. Conf. Comput. Graph. Interact. Techn.*, 1998, pp. 43–54.
- [33] K.-J. Choi and H.-S. Ko, "Stable but responsive cloth," in *Proc. ACM SIGGRAPH*, 2005, pp. 1–8.
- [34] D. Terzopoulos and K. Fleischer, "Modeling inelastic deformation: Viscoelasticity, plasticity, fracture," in *Proc. 15th Annu. Conf. Comput. Graph. Interact. Techn.*, Jun. 1988, pp. 269–278.
- [35] J. Smith, A. Witkin, and D. Baraff, "Fast and controllable simulation of the shattering of brittle objects," *Comput. Graph. Forum*, vol. 20, no. 2, pp. 81–90, Jun. 2001.
- [36] A. Norton, G. Turk, B. Bacon, J. Gerth, and P. Sweeney, "Animation of fracture by physical modeling," *Vis. Comput.*, vol. 7, no. 4, pp. 210–219, Jul. 1991.
- [37] K. Hirota, Y. Tanoue, and T. Kaneko, "Generation of crack patterns with a physical model," *Vis. Comput.*, vol. 14, no. 3, pp. 126–137, Jul. 1998.
- [38] P. Volino and N. Magnenat-Thalmann, "Implementing fast cloth simulation with collision response," in *Proc. Comput. Graph. Int.*, 2000, pp. 257–266.
- [39] R. Bridson, R. Fedkiw, and J. Anderson, "Robust treatment of collisions, contact and friction for cloth animation," in *Proc. 29th Annu. Conf. Comput. Graph. Interact. Techn.*, 2002, pp. 594–603.
- [40] R. Bridson, S. Marino, and R. Fedkiw, "Simulation of clothing with folds and wrinkles," in *Proc. ACM SIGGRAPH Courses*, 2005, p. 3.
- [41] R. Goldenthal, D. Harmon, R. Fattal, M. Bercovier, and E. Grinspun, "Efficient simulation of inextensible cloth," in *Proc. ACM SIGGRAPH Papers*, Jul. 2007, p. 49.
- [42] H. Wang, "A Chebyshev semi-iterative approach for accelerating projective and position-based dynamics," in *Proc. ACM SIGGRAPH*, Oct. 2015, vol. 34, no. 6, pp. 1–9.
- [43] S. Jeong, T.-H. Kim, and C.-H. Kim, "Shrinkage, wrinkling and ablation of burning cloth and paper," *Vis. Comput.*, vol. 27, nos. 6–8, pp. 417–427, Jun. 2011.
- [44] S. Lefebvre and H. Hoppe, "Perfect spatial hashing," *ACM Trans. Graph.*, vol. 25, no. 3, pp. 579–588, Jul. 2006.
- [45] S. Pabst, A. Koch, and W. Straßer, "Fast and scalable CPU/GPU collision detection for rigid and deformable surfaces," *Comput. Graph. Forum*, vol. 29, no. 5, pp. 1605–1612, Jul. 2010.
- [46] X. Faure, F. Zara, F. Jaillet, and J.-M. Moreau, "An implicit tensor-mass solver on the GPU for soft bodies simulation," in *Proc. Eurographics Workshop Virtual Reality Interact. Phys. Simul. (VRIPHYS)*, Dec. 2012, pp. 1–11.
- [47] A. Selle, J. Su, G. Irving, and R. Fedkiw, "Robust high-resolution cloth using parallelism, history-based collisions, and accurate friction," *IEEE Trans. Vis. Comput. Graphics*, vol. 15, no. 2, pp. 339–350, Mar. 2009.
- [48] M. Tang, H. Wang, L. Tang, R. Tong, and D. Manocha, "CAMA: Contact-aware matrix assembly with unified collision handling for GPU-based cloth simulation," *Comput. Graph. Forum*, vol. 35, no. 2, pp. 511–521, May 2016.
- [49] S. Popov, J. Günther, H. Seidel, and P. Slusallek, "Stackless KD-tree traversal for high performance GPU ray tracing," *Comput. Graph. Forum*, vol. 26, no. 3, pp. 415–424, Sep. 2007.
- [50] S. Curtis, R. Tamstorf, and D. Manocha, "Fast collision detection for deformable models using representative-triangles," in *Proc. Symp. Interact. 3D Graph. Games*, Feb. 2008, pp. 61–69.
- [51] H. Delingette, S. Cotin, and N. Ayache, "A hybrid elastic model allowing real-time cutting, deformations and force-feedback for surgery training and simulation," *Proc. Comput. Animation*, vol. 1999, pp. 70–81, May 1999.
- [52] N. Metaaphanon, Y. Bando, B. Chen, and T. Nishita, "Simulation of tearing cloth with frayed edges," *Comput. Graph. Forum*, vol. 28, no. 7, pp. 1837–1844, Oct. 2009.

• • •